Parallel & Distributed Computing

BS CS - Fall Intake 2023

Distributed Web Crawler - Milestone 1

Hamza Ahmed - 29265
Muhammad Hamza Arif - 29234
Laraib Fatima Farooqui - 26521
Arbaaz Murtaza - 29052

1. Repository Details

Primary Repository

- https://github.com/HamzaArif1512/Custom-Distributed-Web-Crawler

  Branch for all Milestone-1 files

- https://github.com/HamzaArif1512/Custom-Distributed-Web-Crawler/tree/M1

  Detailed comparison PDF (on M1 branch)

- https://github.com/HamzaArif1512/Custom-Distributed-Web-Crawler/blob/M1/Web%20Crawler%20Speed%20Comparison.pdf

2. Main Objective

This project aims to design and evaluate a parallel and distributed pipeline for web graph construction (Milestone 1), then use that graph in parallel PageRank computation in later milestones.

For Milestone 1 specifically, the practical objective is:

1. Crawl pages concurrently using Ray-based workers.
2. Extract hyperlinks and construct a directed graph.
3. Avoid duplicate processing through shared or distributed state.
4. Enforce depth/domain constraints for reproducibility.
5. Save graph output in a format suitable for PageRank.

3. About Asyncio

Before moving to distributed Ray execution, the project develops asynchronous crawlers to validate concurrency, link extraction, and crawl-control logic on a single machine.

3.1 `Asyncio/Milestone1_Asyncio.py`

Key lines

Listing 1: Concurrency limiting and async extraction

```
# line 15
domain_semaphores = defaultdict(lambda: asyncio.Semaphore(2))

# line 26
async def extract_links(url, session):
    domain = urlparse(url).netloc
    semaphore = domain_semaphores[domain]
```

Purpose in easy words

- Restricts simultaneous requests to the same domain.
- Fetches HTML pages asynchronously.
- Extracts outgoing hyperlinks and updates graph structure.

Listing 2: Parallel crawling per depth level

```
# line 82
task_list = [extract_links(link, session) for link in links]

# line 85
response = await asyncio.gather(*task_list)
```

Purpose

- Runs many URL fetch tasks concurrently.
- Implements depth-wise expansion of the crawl frontier.

3.2 Asyncio/Milestone1_Asyncio_Mod.py

This is the improved Asyncio design with better controls.

Key lines

Listing 3: Config-driven crawler behavior

```
# lines 27-33
class CrawlConfig:
    num_starting_links: int = 3
    depth: int = 2
    max_urls_per_domain: int = 100
    same_domain_only: bool = False
```

Purpose

- Makes crawl experiments controlled and reproducible.
- Allows easy tuning without rewriting algorithm logic.

Listing 4: Normalization and scope filtering

```
# line 52
def normalize_url(base_url, href):

# line 86
def can_visit_url(url, allowed_domains, config,
    scheduled_per_domain):
    if config.same_domain_only and domain not in
        allowed_domains:
        return False
    if scheduled_per_domain[domain] >= config.
        max_urls_per_domain:
        return False
```

Purpose

- Cleans and standardizes URLs to reduce duplicate variants.
- Enforces domain and per-domain limits.

Listing 5: Concurrent fetch + controlled scheduling

```
# line 185
task_list = [extract_links(link, session, domain_semaphores,
    config, graph, crawled_ok) for link in links]
# line 186
response = await asyncio.gather(*task_list)
# line 195
if not can_visit_url(link, allowed_domains, config,
    scheduled_per_domain):
     continue
```

What the Asyncio phase achieves

- Validates concurrent crawling and graph generation.
- Introduces robust filtering and deterministic constraints.
- Provides a clean bridge to distributed Ray architectures.

4. Ray Implementations and Milestone 1 Key Tasks

4.1 Ray Master-Worker: `Ray/raymasterworker.py`

Key lines

Listing 6: Global parameters and centralized actor

```
# lines 15-17
NUM_WORKERS = 12
DEPTH = 3
MAX_PER_DOMAIN = 50

# line 42
class Master:
```

Listing 7: Shared queue and deduplicated scheduling

```
# line 56
def get_batch(self, batch_size=BATCH_SIZE):

# line 64
def submit_results(self, results):
    for parent_url, depth, child_urls in results:
        ...
        if depth + 1 <= self.max_depth and child not in self.
            visited:
             if self.domain_counts[domain] < MAX_PER_DOMAIN:
                 self.queue.append((child, depth + 1))
```

Listing 8: Parallel extraction and graph persistence

```
# line 91
class WebCrawler:

# line 152
response = await asyncio.gather(...)

# line 209
with open("crawl_graph.json", "w", encoding="utf-8") as f:
```

How this maps to Milestone 1 tasks

1. **Parallel fetch strategy:** multiple Ray workers plus async gather.
2. **Directed graph construction:** parent-child links are inserted in the Master graph.
3. **Duplicate filtering:** global visited set in Master.
4. **Scope limiting: DEPTH** and **MAX_PER_DOMAIN** controls.
5. **PageRank-ready output:** adjacency graph saved as JSON.

4.2 Ray Peer-to-Peer: `Ray/raypeertopeer.py`

Key lines

Listing 9: Partitioned ownership and distributed control

```
# lines 17-19
NUM_WORKERS = 12
DEPTH = 2
MAX_PER_DOMAIN = 50

# line 40
def hash_id(link):
    return int(hashlib.md5(link.encode()).hexdigest(), 16) %
        NUM_WORKERS
```

Listing 10: Worker communication and timing signal

```
# line 47
class CountdownSignal:

# line 65
class WebCrawler:

# line 126
async def recieve(self, links):

# line 132
async def run(self, workers_list, signal):
```

Listing 11: Concurrent crawling, stop checks, and output

```
# line 138
if await signal.is_time_up.remote():
    break

# line 174
chunk_responses = await asyncio.gather(...)

# line 200
await signal.start_countdown.remote(120)

# line 263
with open("crawl_graph.json", "w", encoding="utf-8") as f:
```

How this maps to Milestone 1 tasks

1. Parallel fetch strategy: each worker crawls many URLs concurrently.
2. Directed graph construction: each worker builds local adjacency; graphs are merged.
3. Duplicate filtering: worker-level visited/queued sets and hash-based ownership.
4. Scope limiting: depth checks and per-domain limits.
5. PageRank-ready output: merged adjacency JSON.

4.3 Easy comparison

- **Master-Worker:** simpler correctness and global policy control.
- **Peer-to-Peer:** less central bottleneck, but more coordination complexity.

Together, they provide a strong PDC comparison of centralized vs decentralized workload distribution.

5. Notebook Purpose (Comparison-Focused)

Notebook: `Notebook/WebCrawler.ipynb`

This notebook is mainly for benchmarking and visual comparison among:

- Sequential crawler
- Ray Master-Worker crawler
- Ray Peer-to-Peer crawler

It explicitly reports:

- elapsed time
- URLs crawled
- graph edges
- throughput (URLs/sec)

Hence, its role is experimental evaluation rather than deep architecture explanation. For detailed benchmark interpretation and discussion, refer to the M1-branch PDF:

- https://github.com/HamzaArif1512/Custom-Distributed-Web-Crawler/blob/M1/Web%20Crawler%20Speed%20Comparison.pdf

6. Final Summary

Work completed so far strongly satisfies Milestone 1 objectives:

1. Asyncio implementations establish reliable concurrent crawling and graph extraction.
2. Ray Master-Worker delivers centralized shared-state distributed crawling.
3. Ray Peer-to-Peer delivers decentralized, partitioned crawling.
4. Notebook experiments provide empirical comparison across architectures and parameters.
5. Graph outputs are saved in adjacency-list style JSON suitable for subsequent PageRank stages.

From a CS/PDC perspective, the project already demonstrates:

- task parallelism in a network-bound workload,
- shared-state vs distributed-state coordination trade-offs,
- reproducible crawl-bound policies,
- and performance-oriented evaluation methodology.